

AN INTRODUCTION TO SOFTWARE ARCHITECTURE

1. Introduction	2
2. F2.1. High-level Programming Languages	3
2.2. Abstract Data Types.....	4
2.3. Software Architecture	4
3. Common Architectural Styles.....	5
3.1. Pipes and Filters	6
3.2. Data Abstraction and Object-Oriented Organization	8
3.3. Event-based, Implicit Invocation	9
3.4. Layered Systems	11
3.5. Repositories	12
3.6. Table Driven Interpreters	13
3.7. Other Familiar Architectures.....	14
3.8. Heterogeneous Architectures.....	15
4. Case Studies	16
4.1. Case Study 1: Key Word in Context	16
4.2. Case Study 2: Instrumentation Software	22
4.3. Case 3: A Fresh View of Compilers.....	26
4.4. Case 4: A Layered Design with Different Styles for the Layers	28
4.5. Case 5: An Interpreter Using Different Idioms for the Components.....	30
4.6. Case 6: A Blackboard Globally Recast as Interpreter	33

1. Introduction As the size and complexity of software systems increases, the design problem goes beyond the algorithms and data structures of the computation: designing and specifying the overall system structure emerges as a new kind of problem. Structural issues include gross organization and global control structure; protocols for communication, synchronization, and data access; assignment of functionality to design elements; physical distribution; composition of design elements; scaling and performance; and selection among design alternatives. This is the software architecture level of design.

There is a considerable body of work on this topic, including module interconnection languages, templates and frameworks for systems that serve the needs of specific domains, and formal models of component integration mechanisms. In addition, an implicit body of

work exists in the form of descriptive terms used informally to describe systems. And while there is not currently a well-defined terminology or notation to characterize architectural structures, good software engineers make common use of architectural principles when designing complex software. Many of the principles represent rules of thumb or idiomatic patterns that have emerged informally over time. Others are more carefully documented as industry and scientific standards

It is increasingly clear that effective software engineering requires facility in architectural software design.

First, it is important to be able to recognize common paradigms so that high-level relationships among systems can be understood and so that new systems can be built as variations on old systems.

Second, getting the right architecture is often crucial to the success of a software system design; the wrong one can lead to disastrous results.

Third, detailed understanding of software architectures allows the engineer to make principled choices among design alternatives.

Fourth, an architectural system representation is often essential to the analysis and description of the highlevel properties of a complex system. In this paper we provide an introduction to the field of software architecture. The purpose is to illustrate the current state of the discipline and examine the ways in which architectural design can impact software design. The material presented here is selected from a semester course, Architectures for Software Systems, taught at CMU by the authors . Naturally, a short paper such as this can only briefly highlight the main features of the terrain. This selection emphasizes informal descriptions omitting much of the course's material on specification, evaluation, and selection among design alternatives. We hope, nonetheless, that this will serve to illuminate the nature and significance of this emerging field. In the following section we outline a number of common architectural styles upon which many systems are currently based, and show how heterogeneous styles can be combined in a single design. Next we use six case studies to illustrate how architectural representations of a software system can improve our understanding of complex systems. Finally, we survey some of the outstanding problems in the field, and consider a few of the promising research directions. The text that makes up the bulk of this article has been drawn from numerous other publications by the authors. The taxonomy of architectural styles and

the case studies have incorporated parts of several published papers [1, 2, 3, 4]. To a lesser extent material has been drawn from other articles by the authors [5, 6, 7].

2. From Programming Languages to Software Architecture One characterization of progress in programming languages and tools has been regular increases in abstraction level or the conceptual size of software designers building blocks. To place the field of Software Architecture into perspective let us begin by looking at the historical development of abstraction techniques in computer science.

2.1. High-level Programming Languages When digital computers emerged in the 1950s, software was written in machine language; programmers placed instructions and data individually and explicitly in the computer's memory. Insertion of a new instruction in a program might require hand-checking of the entire program to update references to data and instructions that moved as a result of the insertion. Eventually it was recognized that the memory layout and update of references could be automated, and also that symbolic names could be used for operation codes, and memory addresses. Symbolic assemblers were the result. They were soon followed by macro processors, which allowed a single symbol to stand for a commonly-used sequence of instructions. The substitution of simple symbols for machine operation codes, machine addresses yet to be defined, and sequences of instructions was perhaps the earliest form of abstraction in software. In the latter part of the 1950s, it became clear that certain patterns of execution were commonly useful—indeed, they were so well understood that it was possible to create them automatically from a notation more like mathematics than machine language. The first of these patterns were for evaluation of arithmetic expressions, for procedure invocation, and for loops and conditional statements. These insights were captured in a series of early high-level languages, of which Fortran was the main survivor

Higher-level languages allowed more sophisticated programs to be developed, and patterns in the use of data emerged. Whereas in Fortran data types served primarily as cues for selecting the proper machine instructions, data types in Algol and its successors serve to state the programmer's intentions about how data should be used. The compilers for these languages could build on experience with Fortran and tackle more sophisticated compilation problems. Among other things, they checked adherence to these intentions, thereby providing incentives for the programmers to use the type mechanism.

Progress in language design continued with the introduction of modules to provide protection for related procedures and data structures, with the separation of a module's specification from its implementation, and with the introduction of abstract data types.

2.2. Abstract Data Types In the late 1960s, good programmers shared an intuition about software development: If you get the data structures right, the effort will make development of the rest of the program much easier. The abstract data type work of the 1970s can be viewed as a development effort that converted this intuition into a real theory. The conversion from an intuition to a theory involved understanding • the software structure (which included a representation packaged with its primitive operators),

- specifications (mathematically expressed as abstract models or algebraic axioms),
- language issues (modules, scope, user-defined types),
- integrity of the result (invariants of data structures and protection from other manipulation),
- rules for combining types (declarations),
- information hiding (protection of properties not explicitly included in specifications).

The effect of this work was to raise the design level of certain elements of software systems, namely abstract data types, above the level of programming language statements or individual algorithms. This form of abstraction led to an understanding of a good organization for an entire module that serves one particular purpose. This involved combining representations, algorithms, specifications, and functional interfaces in uniform ways. Certain support was required from the programming language, of course, but the abstract data type paradigm allowed some parts of systems to be developed from a vocabulary of data types rather than from a vocabulary of programming-language constructs.

2.3. Software Architecture

Just as good programmers recognized useful data structures in the late 1960s, good software system designers now recognize useful system organizations.

One of these is based on the theory of abstract data types. But this is not the only way to organize a software system. Many other organizations have developed informally over time, and are now part of the vocabulary of software system designers. For example, typical descriptions of software architectures include synopses such as (*italics ours*):

•“Camelot is based on the client-server model and uses remote procedure calls both locally and remotely to provide communication among applications and servers.”

•“Abstraction layering and system decomposition provide the appearance of system uniformity to clients, yet allow Helix to accommodate a diversity of autonomous devices. The architecture encourages a clientserver model for the structuring of applications.”

•“We have chosen a distributed, object-oriented approach to managing information.”

•“The easiest way to make the canonical sequential compiler into a concurrent compiler is to pipeline the execution of the compiler phases over a number of processors. . . . A more effective way [is to] split the source code into many segments, which are concurrently processed through the various phases of compilation [by multiple compiler processes] before a final, merging pass recombines the object code into a single program.”

Other software architectures are carefully documented and often widely disseminated. Examples include the International Standard Organization's Open Systems Interconnection Reference Model (a layered network architecture), the NIST/ECMA Reference Model (a generic software engineering environment architecture based on layered communication substrates) and the X Window System (a distributed windowed user interface architecture based on event triggering and callbacks). We are still far from having a well-accepted taxonomy of such architectural paradigms, let alone a fully-developed theory of software architecture. But we can now clearly identify a number of architectural patterns, or styles, that currently form the basic repertoire of a software architect.

3. Common Architectural Styles

We now examine some of these representative, broadly-used architectural styles. Our purpose is to illustrate the rich space of architectural choices, and indicate what are some of the tradeoffs in choosing one style over another. To make sense of the differences between styles, it helps to have a common framework from which to view them. The framework we will adopt is to treat an architecture of a specific system as a collection of computational components—or simply components—together with a description of the interactions between these components—the connectors. Graphically speaking, this leads to a view of an abstract architectural description as a graph in which the nodes represent the components and the arcs represent the connectors. As we will see, connectors can represent interactions as varied as procedure call, event broadcast, database queries, and pipes.

An architectural style, then, defines a family of such systems in terms of a pattern of structural organization. More specifically, an architectural style determines the vocabulary of components and connectors that can be used in instances of that style, together with a set of constraints on how they can be combined. These can include topological constraints on

architectural descriptions (e.g., no cycles). Other constraints—say, having to do with execution semantics—might also be part of the style definition.

Given this framework, we can understand what a style is by answering the following questions: What is the structural pattern—the components, connectors, and constraints? What is the underlying computational model? What are the essential invariants of the style? What are some common examples of its use? What are the advantages and disadvantages of using that style? What are some of the common specializations?

3.1. Pipes and Filters

In a pipe and filter style each component has a set of inputs and a set of outputs. A component reads streams of data on its inputs and produces streams of data on its outputs, delivering a complete instance of the result in a standard order. This is usually accomplished by applying a local transformation to the input streams and computing incrementally so output begins before input is consumed. Hence components are termed “filters”. The connectors of this style serve as conduits for the streams, transmitting outputs of one filter to inputs of another. Hence the connectors are termed “pipes”.

Among the important invariants of the style, filters must be independent entities: in particular, they should not share state with other filters. Another important invariant is that filters do not know the identity of their upstream and downstream filters. Their specifications might restrict what appears on the input pipes or make guarantees about what appears on the output pipes, but they may not identify the components at the ends of those pipes. Furthermore, the correctness of the output of a pipe and filter network should not depend on the order in which the filters perform their incremental processing although fair scheduling can be assumed. Figure 1 illustrates this style.

Common specializations of this style include pipelines, which restrict the topologies to linear sequences of filters; bounded pipes, which restrict the amount of data that can reside on a pipe; and typed pipes, which require that the data passed between two filters have a well-defined type.

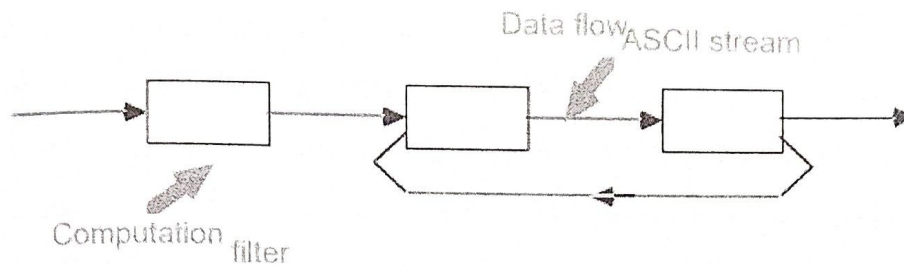


Figure 1: Pipes and Filters

A degenerate case of a pipeline architecture occurs when each filter processes all of its input data as a single entity.¹ In this case the architecture becomes a “batch sequential” system. In these systems pipes no longer serve the function of providing a stream of data, and therefore are largely vestigial. Hence such systems are best treated as instances of a separate architectural style.

The best known examples of pipe and filter architectures are programs written in the Unix shell [16]. Unix supports this style by providing a notation for connecting components (represented as Unix processes) and by providing run time mechanisms for implementing pipes. As another well-known example, traditionally compilers have been viewed as a pipeline systems (though the phases are often not incremental). The stages in the pipeline include lexical analysis, parsing, semantic analysis, code generation. (We return to this example in the case studies.) Other examples of pipes and filters occur in signal processing domains, functional programming, and distributed systems.

Pipe and filter systems have a number of nice properties. First, they allow the designer to understand the overall input/output behavior of a system as a simple composition of the behaviors of the individual filters. Second, they support reuse: any two filters can be hooked together, provided they agree on the data that is being transmitted between them. Third, systems can be easily maintained and enhanced: new filters can be added to existing systems and old filters can be replaced by improved ones. Fourth, they permit certain kinds of specialized analysis, such as throughput and deadlock analysis. Finally, they naturally support concurrent execution. Each filter can be implemented as a separate task and potentially executed in parallel with other filters.

But these systems also have their disadvantages.² First, pipe and filter systems often lead to a batch organization of processing. Although filters can process data incrementally, since filters are inherently independent, the designer is forced to think of each filter as providing a complete transformation of input data to output data. In particular, because of

their transformational character, pipe and filter systems are typically not good at handling interactive applications. This problem is most severe when incremental display updates are required, because the output pattern for incremental updates is radically different from the pattern for filter output. Second, they may be hampered by having to maintain correspondences between two separate, but related streams. Third, depending on the implementation, they may force a lowest common denominator on data transmission, resulting in added work for each filter to parse and unparse its data. This, in turn, can lead both to loss of performance and to increased complexity in writing the filters themselves.

1In general, we find that the boundaries of styles can overlap. This should not deter us from identifying the main features of a style with its central examples of use. 2This is true in spite of the fact that pipes and filters, like every style, has a set of devout religious followers—people who believe that all problems worth solving can best be solved using that particular style.

We haven't mentioned inheritance in this description. While inheritance is an important organizing principle for defining the types of objects in a system, it does not have a direct architectural function. In particular, in our view, an inheritance relationship is not a connector, since it does not define the interaction between components in a system. Also, in an architectural setting inheritance of properties is not restricted to object types but may include connectors and even architectural styles.

3.2. Data Abstraction and Object-Oriented Organization

In this style data representations and their associated primitive operations are encapsulated in an abstract data type or object. The components of this style are the objectsor, if you will, instances of the abstract data types. Objects are examples of a sort of component we call a manager because it is responsible for preserving the integrity of a resource (here the representation). Objects interact through function and procedure invocations. Two important aspects of this style are (a) that an object is responsible for preserving the integrity of its representation (usually by maintaining some invariant over it), and (b) that the representation is hidden from other objects. Figure 2 illustrates this style.³

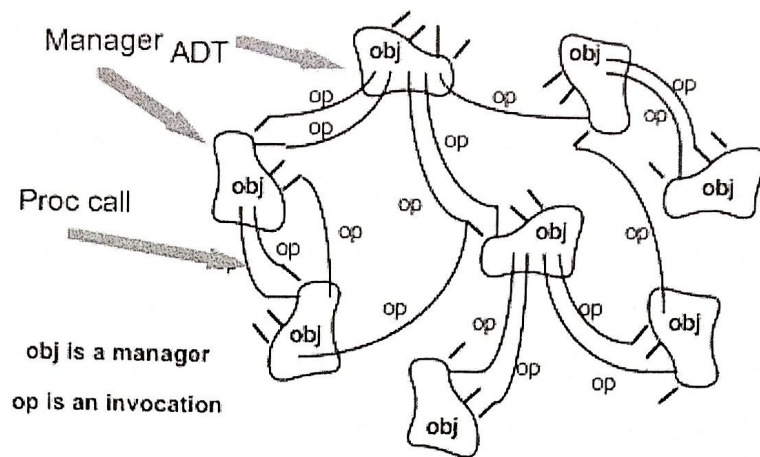


Figure 2: Abstract Data Types and Objects

The use of abstract data types, and increasingly the use of object-oriented systems, is, of course, widespread. There are many variations. For example, some systems allow “objects” to be concurrent tasks; others allow objects to have multiple interfaces [20, 21]. Object-oriented systems have many nice properties, most of which are well known. Because an object hides its representation from its clients, it is possible to change the implementation without affecting those clients. Additionally, the bundling of a set of accessing routines with the data they manipulate allows designers to decompose problems into collections of interacting agents. But object-oriented systems also have some disadvantages. The most significant is that in order for one object to interact with another (via procedure call) it must know the identity of that other object. This is in contrast, for example, to pipe and filter systems, where filters do need not know what other filters are in the system in order to interact with them. The significance of this is that whenever the identity of an object changes it is necessary to modify all other objects that explicitly invoke it. In a module-oriented language this manifests itself as the need to change the “import” list of every module that uses the changed module. Further there can be sideeffect problems: if A uses object B and C also uses B, then C's effects on B look like unexpected side effects to A, and vice versa

3.3. Event-based, Implicit Invocation Traditionally, in a system in which the component interfaces provide a collection of procedures and functions, components interact with each other by explicitly invoking those routines. However, recently there has been considerable interest in an alternative integration technique, variously referred to as implicit invocation, reactive integration, and selective broadcast. This style has historical roots in systems based on actors [22], constraint satisfaction, daemons, and packet-switched networks. The idea behind implicit invocation is that instead of invoking a procedure directly, a component can

announce (or broadcast) one or more events. Other components in the system can register an interest in an event by associating a procedure with the event. When the event is announced the system itself invokes all of the procedures that have been registered for the event. Thus an event announcement ``implicitly" causes the invocation of procedures in other modules. For example, in the Field system [23], tools such as editors and variable monitors register for a debugger's breakpoint events. When a debugger stops at a breakpoint, it announces an event that allows the system to automatically invoke methods in those registered tools. These methods might scroll an editor to the appropriate source line or redisplay the value of monitored variables. In this scheme, the debugger simply announces an event, but does not know what other tools (if any) are concerned with that event, or what they will do when that event is announced.

Architecturally speaking, the components in an implicit invocation style are modules whose interfaces provide both a collection of procedures (as with abstract data types) and a set of events. Procedures may be called in the usual way. But in addition, a component can register some of its procedures with events of the system. This will cause these procedures to be invoked when those events are announced at run time. Thus the connectors in an implicit invocation system include traditional procedure call as well as bindings between event announcements and procedure calls.

The main invariant of this style is that announcers of events do not know which components will be affected by those events. Thus components cannot make assumptions about order of processing, or even about what processing, will occur as a result of their events. For this reason, most implicit invocation systems also include explicit invocation (i.e., normal procedure call) as a complementary form of interaction.

Examples of systems with implicit invocation mechanisms abound [7]. They are used in programming environments to integrate tools [23, 24], in database management systems to ensure consistency constraints [22, 25], in user interfaces to separate presentation of data from applications that manage the data [26, 27], and by syntax-directed editors to support incremental semantic checking.

One important benefit of implicit invocation is that it provides strong support for reuse. Any component can be introduced into a system simply by registering it for the events of that system. A second benefit is that implicit invocation eases system evolution [30]. Components may be replaced by other components without affecting the interfaces of other components in the system. In contrast, in a system based on explicit invocation, whenever the identity of a that provides some system function is changed, all other modules that import that

module must also be changed. The primary disadvantage of implicit invocation is that components relinquish control over the computation performed by the system. When a component announces an event, it has no idea what other components will respond to it. Worse, even if it does know what other components are interested in the events it announces, it cannot rely on the order in which they are invoked. Nor can it know when they are finished. Another problem concerns exchange of data. Sometimes data can be passed with the event. But in other situations event systems must rely on a shared repository for interaction. In these cases global performance and resource management can become a serious issue. Finally, reasoning about correctness can be problematic, since the meaning of a procedure that announces events will depend on the context of bindings in which it is invoked. This is in contrast to traditional reasoning about procedure calls, which need only consider a procedure's pre- and post-conditions when reasoning about an invocation of it.

3.4. Layered Systems

A layered system is organized hierarchically, each layer providing service to the layer above it and serving as a client to the layer below. In some layered systems inner layers are hidden from all except the adjacent outer layer, except for certain functions carefully selected for export. Thus in these systems the components implement a virtual machine at some layer in the hierarchy. (In other layered systems the layers may be only partially opaque.) The connectors are defined by the protocols that determine how the layers will interact. Topological constraints include limiting interactions to adjacent layers. Figure 3 illustrates this style

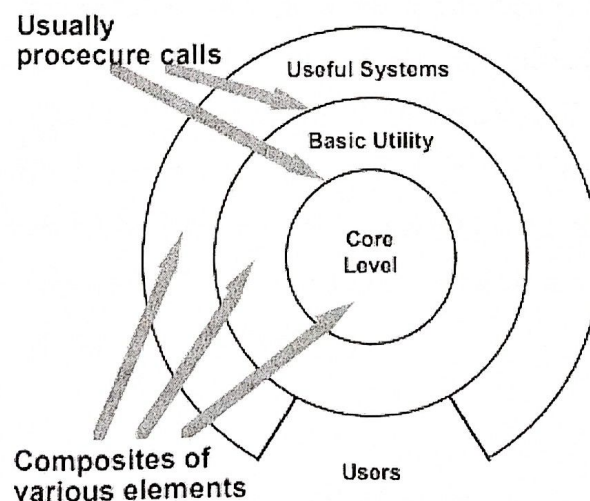


Figure 3: Layered Systems

The most widely known examples of this kind of architectural style are layered communication protocols [31]. In this application area each layer provides a substrate for

communication at some level of abstraction. Lower levels define lower levels of interaction, the lowest typically being defined by hardware connections. Other application areas for this style include database systems and operating systems.

Layered systems have several desirable properties. First, they support design based on increasing levels of abstraction. This allows implementors to partition a complex problem into a sequence of incremental steps. Second, they support enhancement. Like pipelines, because each layer interacts with at most the layers below and above, changes to the function of one layer affect at most two other layers. Third, they support reuse. Like abstract data types, different implementations of the same layer can be used interchangeably, provided they support the same interfaces to their adjacent layers. This leads to the possibility of defining standard layer interfaces to which different implementors can build. (A good example is the OSI ISO model and some of the X Window System protocols.)

But layered systems also have disadvantages. Not all systems are easily structured in a layered fashion. (We will see an example of this later in the case studies.) And even if a system can logically be structured as layers, considerations of performance may require closer coupling between logically high-level functions and their lower-level implementations. Additionally, it can be quite difficult to find the right levels of abstraction. This is particularly true for standardized layered models. One notes that the communications community has had some difficulty mapping existing protocols into the ISO framework: many of those protocols bridge several layers.

In one sense this is similar to the benefits of implementation hiding found in abstract data types. However, here there are multiple levels of abstraction and implementation. They are also similar to pipelines, in that components communicate at most with one other component on either side. But instead of simple pipe read/write protocol of pipes, layered systems can provide much richer forms of interaction. This makes it difficult to define system-independent layers (as with filters)—since a layer must support the specific protocols at its upper and lower boundaries. But it also allows much closer interaction between layers, and permits two-way transmission of information.

3.5. Repositories

In a repository style there are two quite distinct kinds of components: a central data structure represents the current state, and a collection of independent components operate on the central data store. Interactions between the repository and its external components can vary significantly between systems.

The choice of control discipline leads to major subcategories. If the types of transactions in an input stream of transactions trigger selection of processes to execute, the repository can be a traditional database. If the current state of the central data structure is the main trigger of selecting processes to execute, the repository can be a blackboard.

Figure 4 illustrates a simple view of a blackboard architecture. (We will examine more detailed models in the case studies.) The blackboard model is usually presented with three major parts:

1. The knowledge sources: separate, independent parcels of application-dependent knowledge. Interaction among knowledge sources takes place solely through the blackboard.
2. The blackboard data structure: problem-solving state data, organized into an application-dependent hierarchy. Knowledge sources make changes to the blackboard that lead incrementally to a solution to the problem.
3. Control: driven entirely by state of blackboard. Knowledge sources respond opportunistically when changes in the blackboard make them applicable.

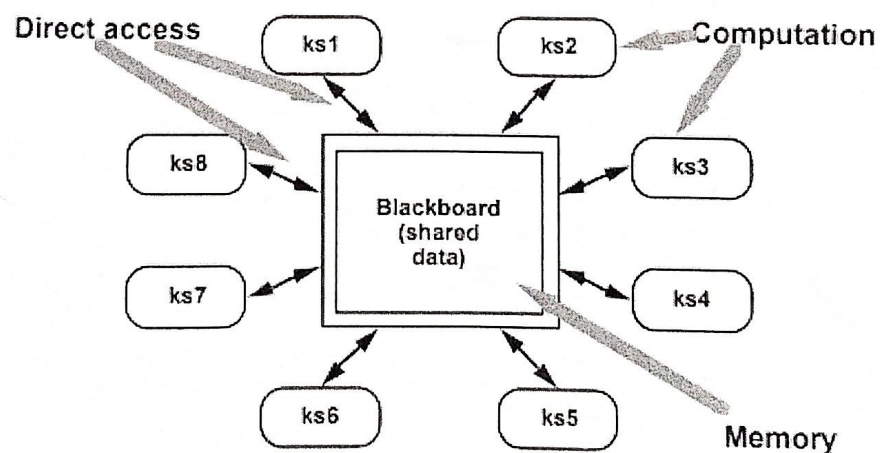


Figure 4: The Blackboard

In the diagram there is no explicit representation of the control component. Invocation of a knowledge source is triggered by the state of the blackboard. The actual locus of control, and hence its implementation, can be in the knowledge sources, the blackboard, a separate module, or some combination of these.

Blackboard systems have traditionally been used for applications requiring complex interpretations of signal processing, such as speech and pattern recognition.

Several of these are surveyed by Nii [34]. They have also appeared in other kinds of systems that involve shared access to data with loosely coupled agents.

There are, of course, many other examples of repository systems. Batchsequential systems with global databases are a special case. Programming environments are often organized as a collection of tools together with a shared repository of programs and program fragments [36]. Even applications that have been traditionally viewed as pipeline architectures, may be more accurately interpreted as repository systems. For example, as we will see later, while a compiler architecture has traditionally been presented as a pipeline, the “phases” of most modern compilers operate on a base of shared information (symbol tables, abstract syntax tree, etc.).

3.6. Table Driven Interpreters

In an interpreter organization a virtual machine is produced in software. An interpreter includes the pseudo-program being interpreted and the interpretation engine itself. The pseudo-program includes the program itself and the interpreter’s analog of its execution state (activation record). The interpretation engine includes both the definition of the interpreter and the current state of its execution. Thus an interpreter generally has four components: an interpretation engine to do the work, a memory that contains the pseudo-code to be interpreted, a representation of the control state of the interpretation engine, and a representation of the current state of the program being simulated. (See Figure 5.)

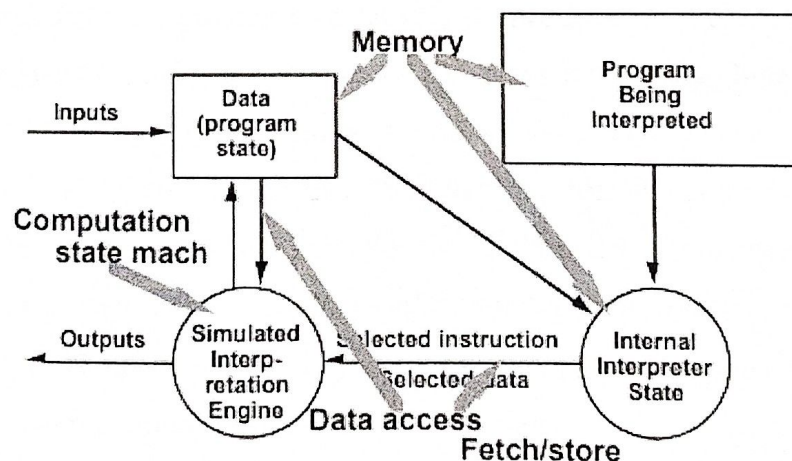


Figure 5: Interpreter

Interpreters are commonly used to build virtual machines that close the gap between the computing engine expected by the semantics of the program and the computing engine available in hardware. We occasionally speak of a programming language as providing, say, a “virtual Pascal machine.” We will return to interpreters in more detail in the case studies. 3.7. Other Familiar Architectures There are numerous other architectural styles and patterns.

Some are widespread and others are specific to particular domains. While a complete treatment of these is beyond the scope of this paper, we briefly note a few of the important categories.

- **Distributed processes:** Distributed systems have developed a number of common organizations for multi-process systems [37]. Some can be characterized primarily by their topological features, such as ring and star organizations. Others are better characterized in terms of the kinds of inter-process protocols that are used for communication (e.g., heartbeat algorithms). One common form of distributed system architecture is a “client-server” organization [38]. In these systems a server represents a process that provides services to other processes (the clients). Usually the server does not know in advance the identities or number of clients that will access it at run time. On the other hand, clients know the identity of a server (or can find it out through some other server) and access it by remote procedure call.

- **Main program/subroutine organizations:** The primary organization of many systems mirrors the programming language in which the system is written. For languages without support for modularization this often results in a system organized around a main program and a set of subroutines. The main program acts as the driver for the subroutines, typically providing a control loop for sequencing through the subroutines in some order.
- **Domain-specific software architectures:** Recently there has been considerable interest in developing “reference” architectures for specific domains [39]. These architectures provide an organizational structure tailored to a family of applications, such as avionics, command and control, or vehicle management systems. By specializing the architecture to the domain, it is possible to increase the descriptive power of structures. Indeed, in many cases the architecture is sufficiently constrained that an executable system can be generated automatically or semi-automatically from the architectural description itself.
- **State transition systems:** A common organization for many reactive systems is the state transition system. These systems are defined in terms of a set of states and a set of named transitions that move a system from one state to another.
- **Process control systems:** Systems intended to provide dynamic control of a physical environment are often organized as process control systems. These systems are roughly characterized as a feedback loop in which inputs from sensors are used by the process control system to determine a set of outputs that will produce a new state of the environment.

3.8. Heterogeneous Architectures Thus far we have been speaking primarily of “pure” architectural styles. While it is important to understand the individual nature of each of these styles, most systems typically involve some combination of several styles.

There are different ways in which architectural styles can be combined. One way is through hierarchy. A component of a system organized in one architectural style may have an internal structure that is developed a completely different style. For example, in a Unix pipeline the individual components may be represented internally using virtually any style including, of course, another pipe and filter, system.

What is perhaps more surprising is that connectors, too, can often be hierarchically decomposed. For example, a pipe connector may be implemented internally as a FIFO queue accessed by insert and remove operations.

A second way for styles to be combined is to permit a single component to use a mixture of architectural connectors. For example, a component might access a repository through part of its interface, but interact through pipes with other components in a system, and accept control information through another part of its interface. (In fact, Unix pipe and filter systems do this, the file system playing the role of the repository and initialization switches playing the role of control.)

Another example is an “active database”. This is a repository which activates external components through implicit invocation. In this organization external components register interest in portions of the database. The database automatically invokes the appropriate tools based on this association. (Blackboards are often constructed this way; knowledge sources are associated with specific kinds of data, and are activated whenever that kind of data is modified.)

A third way for styles to be combined is to completely elaborate one level of architectural description in a completely different architectural style. We will see examples of this in the case studies.

4. Case Studies

We now present six examples to illustrate how architectural principles can be used to increase our understanding of software systems. The first example shows how different architectural solutions to the same problem provide different benefits. The second case study summarizes experience in developing a domain-specific architectural style for a family of industrial products. The third case study examines the familiar compiler architecture in a

fresh light. The remaining three case studies present examples of the use of heterogeneous architectures.

4.1. Case Study 1:

Key Word in Context In his paper of 1972, Parnas proposed the following problem: The KWIC [Key Word in Context] index system accepts an ordered set of lines, each line is an ordered set of words, and each word is an ordered set of characters. Any line may be "circularly shifted" by repeatedly removing the first word and appending it at the end of the line. The KWIC index system outputs a listing of all circular shifts of all lines in alphabetical order

Parnas used the problem to contrast different criteria for decomposing a system into modules. He describes two solutions, one based on functional decomposition with shared access to data representations, and a second based on a decomposition that hides design decisions. Since its introduction, the problem has become well-known and is widely used as a teaching device in software engineering. Garlan, Kaiser, and Notkin also use the problem to illustrate modularization schemes based on implicit invocation.

While KWIC can be implemented as a relatively small system it is not simply of pedagogical interest. Practical instances of it are widely used by computer scientists. For example, the "permuted" [sic] index for the Unix Man pages is essentially such a system.

From the point of view of software architecture, the problem derives its appeal from the fact that it can be used to illustrate the effect of changes on software design.

Parnas shows that different problem decompositions vary greatly in their ability to withstand design changes. Among the changes he considers are:

- Changes in processing algorithm: For example, line shifting can be performed on each line as it is read from the input device, on all the lines after they are read, or on demand when the alphabetization requires a new set of shifted lines.
- Changes in data representation: For example, lines can be stored in various ways. Similarly, circular shifts can be stored explicitly or implicitly (as pairs of index and offset).
- Enhancement to system function: For example, modify the system so that shifted lines to eliminate circular shifts that start with certain noise words (such as "a", "an", "and", etc.). Change the system to be interactive, and allow the user to delete lines from the original (or, alternatively, from circularly shifted) lists.
- Performance: Both space and time.
- Reuse: To what extent can the components serve as reusable entities.

We now outline four architectural designs for the KWIC system. All four are grounded in published solutions (including implementations). The first two are those considered in Parnas' original article. The third solution is based on the use of an implicit invocation style and represents a variant on the solution examined by Garlan, Kaiser, and Notkin. The fourth is a pipeline solution inspired by the Unix index utility.

After presenting each solution and briefly summarizing its strengths and weakness, we contrast the different architectural decompositions in a table organized along the five design dimensions itemized above.

Solution 1: Main Program/Subroutine with Shared Data

The first solution decomposes the problem according to the four basic functions performed: input, shift, alphabetize, and output. These computational components are coordinated as subroutines by a main program that sequences through them in turn. Data is communicated between the components through shared storage ("core storage"). Communication between the computational components and the shared data is an unconstrained readwrite protocol. This is made possible by the fact that the coordinating program guarantees sequential access to the data. (See Figure 6.)

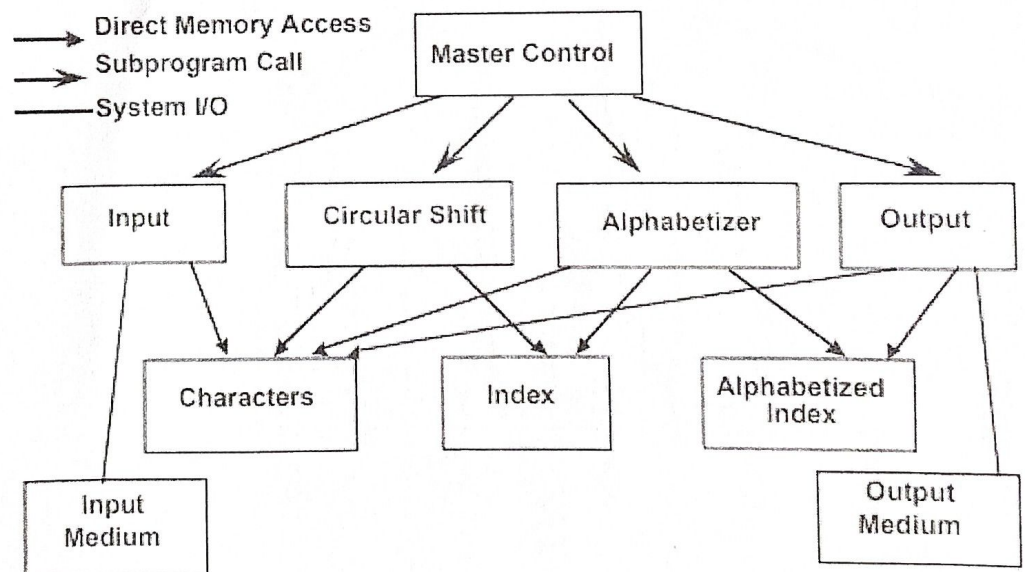


Figure 6: KWIC - Shared Data Solution

Using this solution data can be represented efficiently, since computations can share the same storage. The solution also has a certain intuitive appeal, since distinct computational aspects are isolated in different modules.

However, , it has a number of serious drawbacks in terms of its ability to handle changes. In particular, a change in data storage format will affect almost all of the modules.

Similarly changes in the overall processing algorithm and enhancements to system function are not easily accommodated. Finally, this decomposition is not particularly supportive of reuse.

Solution 2: Abstract Data Types The second solution decomposes the system into a similar set of five modules. However, in this case data is no longer directly shared by the computational components. Instead, each module provides an interface that permits other components to access data only by invoking procedures in that interface. (See Figure 7, which illustrates how each of the components now has a set of procedures that determine the form of access by other components in the system.)

This solution provides the same logical decomposition into processing modules as the first. However, it has a number of advantages over the first solution when design changes are considered. In particular, both algorithms and data representations can be changed in individual modules without affecting others. Moreover, reuse is better supported than in the first solution because modules make fewer assumptions about the others with which they interact.

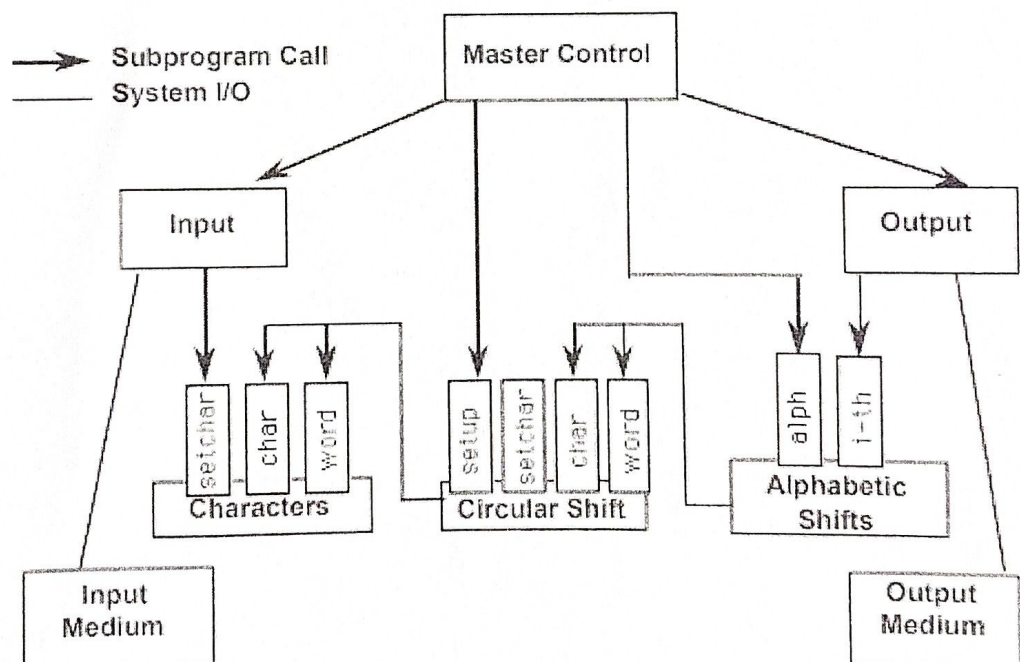


Figure 7: KWIC - Abstract Data Type Solution

On the other hand, as discussed by Garlan, Kaiser, and Notkin, the solution is not particularly well-suited to enhancements. The main problem is that to add new functions to the system, the implementor must either modify the existing modules—compromising their simplicity and integrity—or add new modules that lead to performance penalties.

SOLUTION 3: IMPLICIT INVOCATION

The third solution uses a form of component integration based on shared data similar to the first solution. However, there are two important differences. First, the interface to the data is more abstract. Rather than exposing the storage formats to the computing modules, data is accessed abstractly (for example, as a list or a set). Second, computations are invoked implicitly as data is modified. Thus interaction is based on an active data model. For example, the act of adding a new line to the line storage causes an event to be sent to the shift module. This allows it to produce circular shifts (in a separate abstract shared data store). This in turn causes the alphabetizer to be implicitly invoked so that it can alphabetize the lines.

This solution easily supports functional enhancements to the system: additional modules can be attached to the system by registering them to be invoked on data-changing events. Because data is accessed abstractly, it also insulates computations from changes in data representation. Reuse is also supported, since the implicitly invoked modules only rely on the existence of certain externally triggered events.

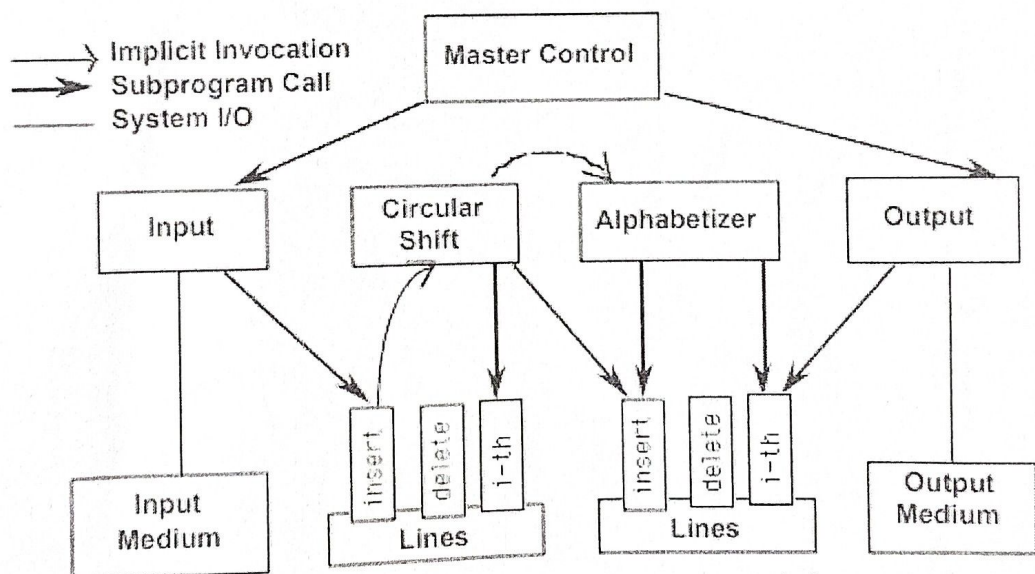


Figure 8: KWIC - Implicit Invocation Solution

However, the solution suffers from the fact that it can be difficult to control the order of processing of the implicitly invoked modules. Further, because invocations are data driven, the most natural implementations of this kind of decomposition tend to use more space than the previously considered decompositions.